

AER1217: Development of Unmanned Aerial Vehicles

Project:

Autonomous Drone Racing Competition

Group 2:

Zouhair Adam Hamaimou (Student #1004891986)

Mark Zhuang (Student #1006963644)

Yi Lo (Student #1009764550)

April 28th, 2025

1 Algorithm Overview and Justification (7%)

We implement a four-stage pipeline:

1. **Occupancy grid construction:** Discretize $[-3.5, 3.5]^2$ at 0.1 m resolution; mark cells within obstacle radius $r_{\text{obs}} = 0.3$ m as occupied.
2. **Gate-corridor carving:** For each gate center (g_x, g_y) and yaw ψ , enforce a safe corridor of half-width $w/2 = 0.2$ m by marking all cells outside this corridor as occupied.
3. **Segmented A* search:** Plan pairwise between waypoints with entry and exit offsets (± 0.3 m) to each gate, and seal off the gate area after passing through. Use the cost function:

$$f(n) = g(n) + h(n), \quad g(n) = \sum_{\text{steps}} \|\Delta \mathbf{x}\|, \quad h(n) = \|\mathbf{x} - \mathbf{x}_{\text{goal}}\|$$

on an 8-connected grid.

4. **Spline smoothing and enforcement:** Densify to 0.2 m spacing, lift to $z = 1$ m, fit B-splines combined with slerp interpolation to allow slight waypoint deviations while maintaining smoothness. Enforce dynamic constraints:

$$\kappa(s) = \frac{|x' y'' - y' x''|}{(x'^2 + y'^2)^{3/2}}, \quad v(s) \leq \min(v_{\text{max}}, \sqrt{a_{\text{max}}/\kappa})$$

Resampled at 30 Hz.

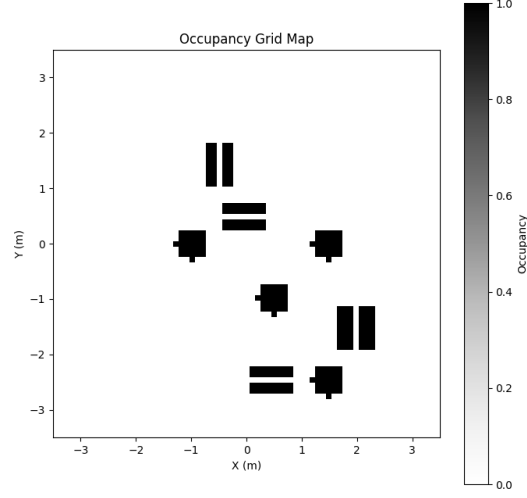


Figure 1: Occupancy grid after obstacle inflation and gate-corridor carving.

Planner Pseudocode

Algorithm 1 Segmented A* with Gate Constraints

```

1: Input: obstacle centers, gate centers & yawes, start, final target
2: Build zeroed grid over  $[-3.5, 3.5]^2$ 
3: for each obstacle do
4:   Inflate radius  $r_{\text{obs}} = 0.3$ , mark grid cells occupied
5: end for
6: for each gate do
7:   Carve corridor: mark lateral cells outside  $\pm 0.2$  m as occupied
8: end for
9: Define race sequence: [gate1, gate3, gate4, gate2, gate1, gate4]
10: Generate entry/exit offsets  $\pm 0.3$  m per gate
11:  $W \leftarrow [\text{start}, \{\text{entry, center, exit}\}_{\text{gates}}, \text{target}]$ 
12:  $P \leftarrow \emptyset$ 
13: for  $i = 1$  to  $|W| - 1$  do
14:    $p \leftarrow \text{AStar}(W[i], W[i + 1], \text{grid})$ 
15:   if  $\|p_{\text{end}} - W[i + 1]\| > 0.05$  then
16:     append  $W[i + 1]$ 
17:   end if
18:   densify( $p$ );  $P \leftarrow P \cup p$ 
19:   block_behind_corridor( $W[i + 1]$ )
20: end for
21: smooth_PCHIP_with_slarp( $P$ )  $\rightarrow (\mathbf{r}(t), \dot{\mathbf{r}}(t), \ddot{\mathbf{r}}(t))$ 

```

2 Simulation Results (5%)

We ran three firmware-mode trials on the Myhal course and achieved **1st place** overall.

Table 1: Lap times under different dynamic caps

Trial	$v_{\max}$ (xy)	$a_{\max}$ (xy)	Flight time (s)
1	1.0 m/s	4.0 m/s ²	44.00
2	1.2 m/s	6.0 m/s ²	27.50
3	1.5 m/s	10.0 m/s ²	19.42

All four gates were passed without collision; hover target reached within 0.15 m.

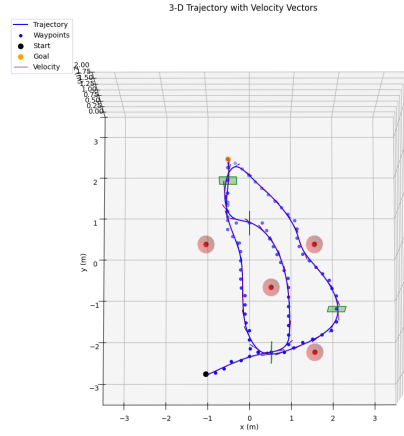


Figure 2: Top-down view of the 3D trajectory with velocity vectors.

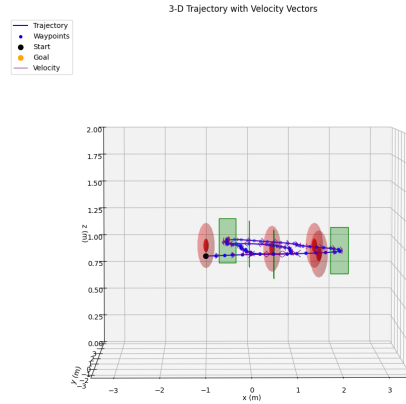


Figure 3: Side view showing smooth altitude profile.

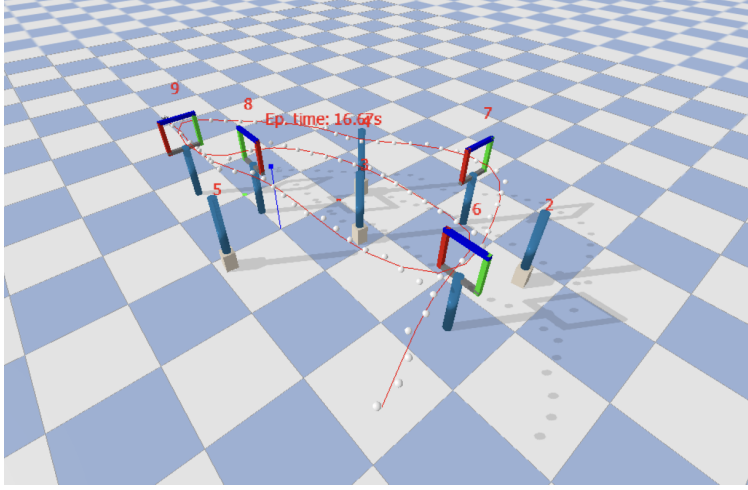


Figure 4: Simulation snapshot: white spheres are densified waypoints; red line is the spline.

3 Difficulties & Lessons Learned (5%)

- Tuning maximum velocity and acceleration to match real hardware limits.
- Tuning grid resolution for safe navigation versus computational efficiency.
- Tuning allowed deviations of the spline from waypoints during smoothing.
- Corridor carving parameters: fine-tuning lateral clearance to ensure passability.
- PCHIP spline initially violating $a_{\max}$ in curves; iterative time-stretch used.
- Firmware streaming latency ($\sim 0.5\text{s}$) requiring a buffered takeoff.

4 Improvements & Next Steps (3%)

- **RPY rates optimization:** Tailoring roll-pitch-yaw rates for tighter turns.
- **Smoother (diagonal) takeoff and landing:** More natural, efficient trajectories.
- **Dynamic replanning:** Reinvoke A* search if position error exceeds 0.2 m.
- **Hierarchical grid planning:** Coarse global planning, fine local refinement.
- **Yaw alignment:** Align drone heading with $\text{atan2}(\dot{y}, \dot{x})$ for aerodynamic efficiency.

References

- 1 UTIAS, *Safe-Control-Gym Documentation*, 2024.
- 2 CrazySwarm API Documentation. Available at: <https://crazyswarm.readthedocs.io/>.
- 3 P. Schlageter, T. Lin, A. Kuo, and A. Kapoor, “Curvature-Aware Trajectory Generation,” *IEEE Transactions on Robotics*, 2023.